Kubernetes

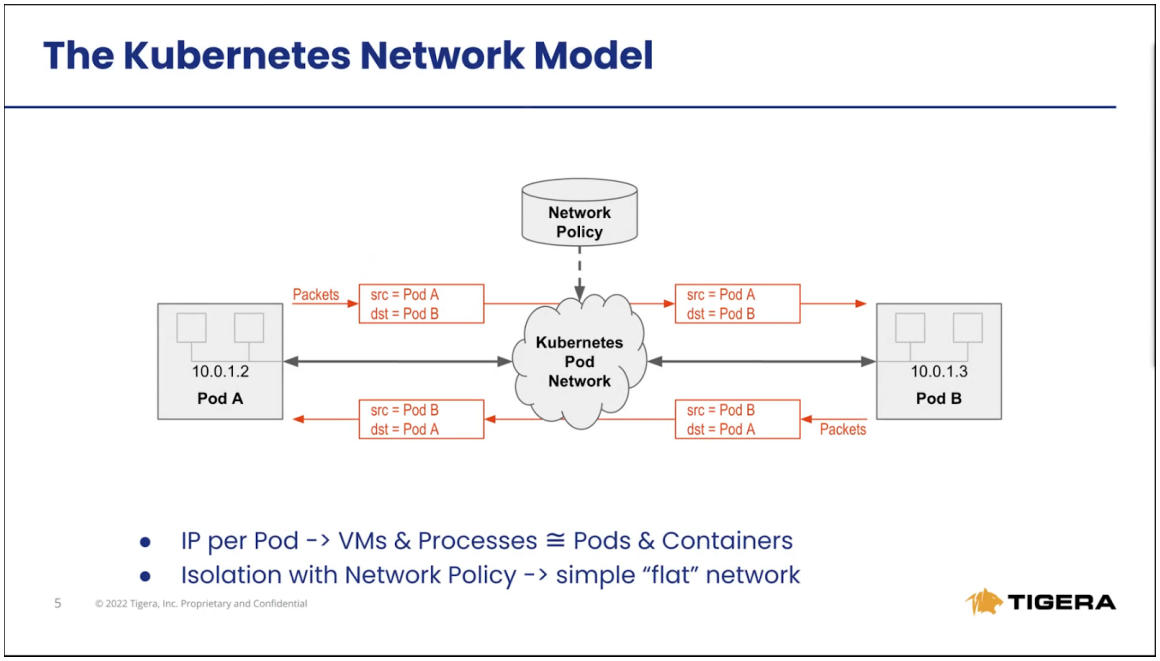
How to secure your Kubernetes Container Network Interface (CNI) with Calico

Networking is a vital component of any Kubernetes setup, but Kubernetes itself has very few prescribed networking conditions. The Kubernetes network model is itself designed to be low-friction with minimal complexity, which is fine if you're just getting started - but if you want to achieve a true zero-trust network, then you'll need to look at installing a third-party solution.

Project Calico is one such solution. The Calico network policy was designed to work seamlessly with Kubernetes, expanding the existing network policy rules to cover multiple types of endpoints.

In March, we were joined by Chris Tomkins, Lead Developer Advocate at Tigera, the primary contributor and owner of Project Calico. Chris discussed Kubernetes network policies, integrating the Calico network policy and container network interfaces (CNIs), and demonstrated how to use Project Calico to secure your network.

The Kubernetes network model



The network conditions that Kubernetes prescribes are very basic. Each pod is given its own IP address, and containers within those pods share that address. There are no

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

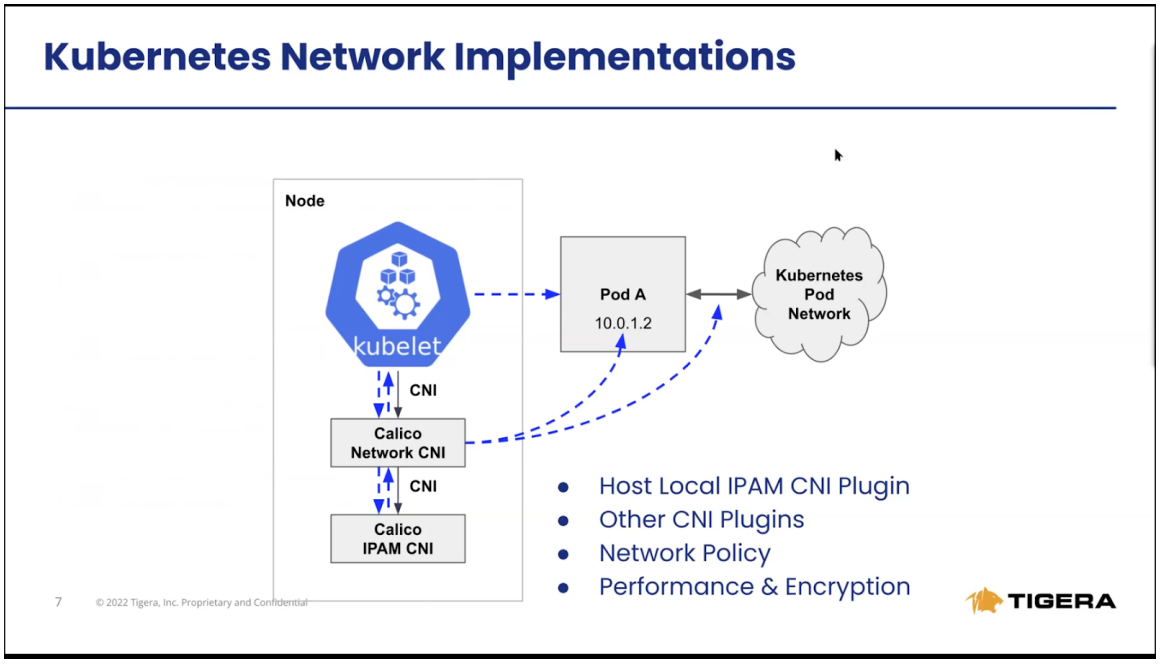
network restrictions within each pod, so containers can communicate with each other freely. Similarly, every pod can automatically communicate with all other pods in the cluster. The only restriction here is that this communication must be done on a layer 3 routed network and no network address translation is allowed between pods.

However, this isn't ideal from a security perspective. Organizations don't typically want workloads to talk to each other with no controls. Modern security best practices involve zero-trust networks where communication between pods and clusters only happens if it's absolutely necessary.

With that in mind, Kubernetes needs additional network policies that define how that communication flows. However, network policies operate differently in Kubernetes. Chris explains:

"We don't sort by IP addresses because they're ephemeral - they come and go quickly - so we apply network policy between the pods in Kubernetes. But network policy isn't actually implemented in Kubernetes. It's typically left to the CNI to handle."

This allows for organizations to introduce isolation into their networks. Given that Kubernetes is set up as a flat network to allow everything to talk to each other, additional network policies are needed to narrow that communication down to keep everything secure.



Each Kubernetes node has an agent called kubelet, which is responsible for scheduling pods on each node. Whenever it needs to bring up a new pod, it will send traffic through the CNI plugin. As the example shows, CNI plugins can also be chained together to provide additional functionality. For example, one plugin could be responsible for network connectivity, while another would cover IP address management (IPAM).

Here, Chris explains why Kubernetes doesn't handle more advanced networking protocols:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

“By abstracting the network layer, Kubernetes can focus on what it’s good at. That’s why it allows networking vendors, open-source communities, and anyone else who’s interested to build a CNI provided it conforms to the Kubernetes network model.”

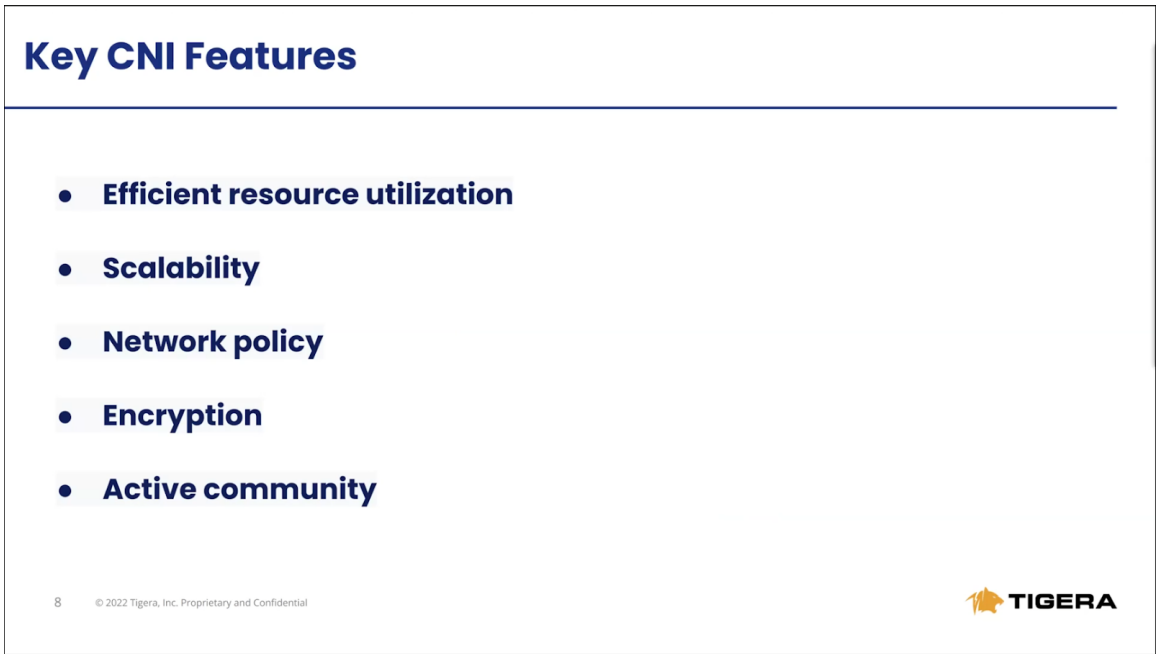
This is where Calico comes in. Calico provides network connectivity and IP address management plugins, as well as an additional network policy that sits on top of the one provided by Kubernetes.

So, in this example, Kubernetes tells Kubelet to set up a new pod. This calls up the Calico network CNI to set up the network, and that in turn calls the Calico IPAM plugin to provide an IP address. That IP address is returned to the first Calico plugin, which then passes the information back to Kubelet to set up the new pod on the network.

Organizations don’t have to choose between Calico and other CNI plugins, either. Chris says:

“Calico’s flexible design allows it to be set up with a range of other CNI plugins. There are other CNI plugins available, for example, Amazon and Azure Cloud both have their own plugins that can be used alongside Calico. Calico’s happy to run alongside others, so you’re not limited to one vendor”

Implementing CNIs to a Kubernetes network



There’s no one answer for choosing a certain CNI over another, though. Organizations might choose to use a certain CNI because it’s mandated by the Kubernetes provider or they might not feel comfortable making changes to the existing network structure. On the other hand, they might need a certain CNI because they have a bare cluster that needs additional networking functionality that works across multiple environments - which is something that Calico offers.

The slide above shows what features are considered core to a good CNI.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Organizations should look for efficient performance, as this means traffic will be forwarded quickly with very little CPU consumption. A CNI should also be able to scale to the same number of nodes that Kubernetes can handle overall to avoid performance bottlenecks.

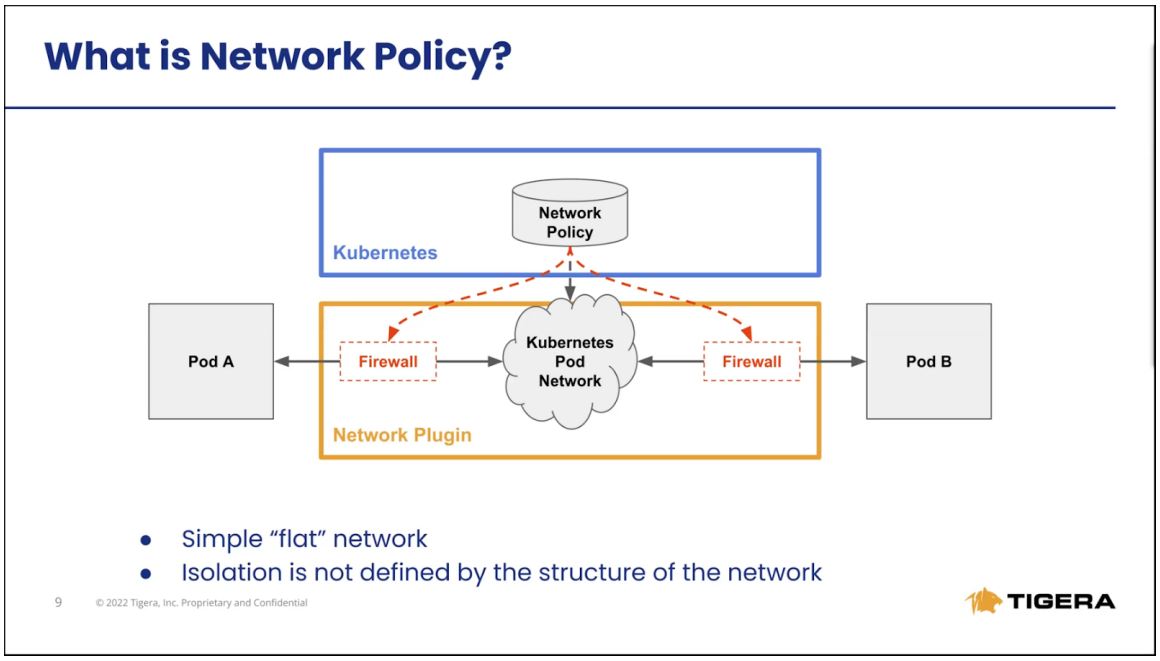
It's also important to have a CNI with the original reference implementation of the Kubernetes network policy. Chris adds:

"Not every CNI implements network policy, so if you created a network policy object, it just wouldn't work. That can be pretty dangerous because, if you have network policy objects that restrict access to certain parts of the cluster and your CNI doesn't implement network policy, everything would be open."

CNIs should also ideally provide encryption to protect inter-node and inter-pod traffic in the network. Calico supports the Wireguard and IPsec encryption protocols, depending on the data plane that's inserted.

Finally, an active community is vital to the health of a CNI. All parts of Kubernetes thrive on the features and innovation that come with community and company contributors, and CNIs are no different.

Understanding network policy



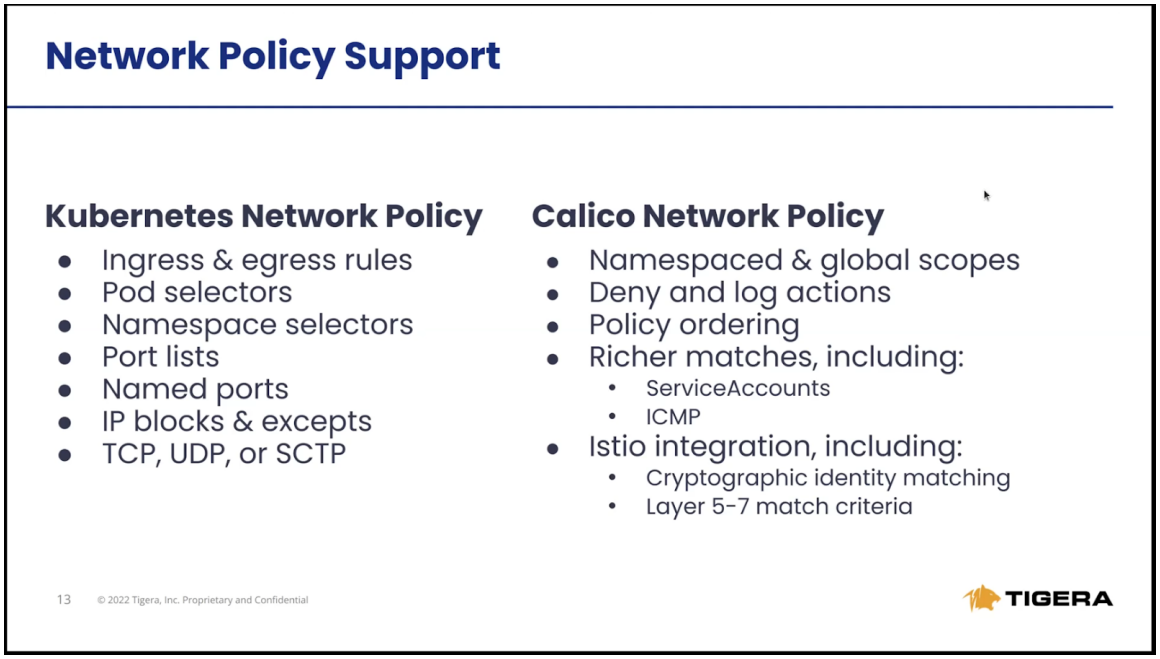
The way that network policy works with Kubernetes is as a layer on top of the defined flat network. While this might initially sound limiting, it's far easier to implement and scale than traditional systems. As Chris puts it:

"In the old days, your policy would be defined by the physical topology of the network. And, any devices you added would have to perform at a really high level because they were often in real chokepoints in the network. However, Kubernetes is much more scaleable. You define network topology once in the Kubernetes API, and it's applied by the CNI plugins as close as possible to the pods in the cluster."

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

In effect, this means that each pod has a virtual firewall between it and the rest of the network. With Kubernetes, network policy and isolations aren't defined by the layout of a network, giving organizations more scope to apply network policy and make changes as the network grows.

Similarly, because firewalls are run by the same host as Kubernetes pods, it's a lot easier to scale the network. Only a small percentage of CPU is used to manage network policy so there's no requirement to purchase a large firewall upfront as there often is with physical servers.



Given that the native Kubernetes network policy is extremely basic, Calico's network policy was designed to provide additional functionality, making it operate more like a traditional firewall. A significant part of this is the use of policy ordering.

Policy ordering allows certain teams within an organization to administer network policy within their respective namespaces, but it leaves overall control with the security team. By defining an order for policies to be enacted, security teams can administer the front and back end of the security stack and make sweeping rules while allowing some teams a degree of control over their own environments.

Enhancing Kubernetes network policy with Calico

With Kubernetes having very few prescribed network conditions, organizations will need to implement additional network policies, container network interfaces, and security policies to keep their Kubernetes networks secure. One way of doing this is by installing the Calico network policy and CNIs.

Calico's network tools work seamlessly with Kubernetes, providing an extension of the pre-existing network policies that can be enforced through Calico CNIs. Whether used alone or in tangent with CNIs from other vendors, Calico's network tools offer a portable security solution that works regardless of which vendor provides Kubernetes for you.

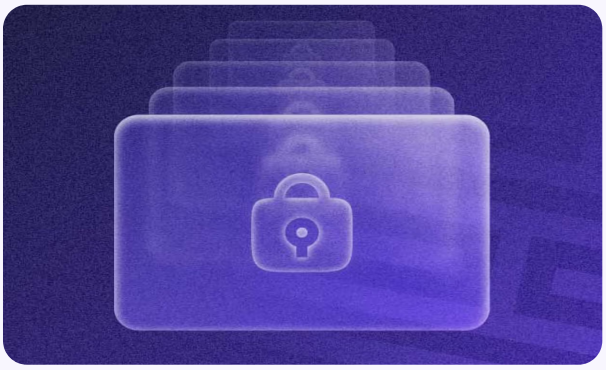
Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Thanks again to Chris Tomkins for this insightful webinar. If you haven't already, make sure to watch the webinar recap as Chris demonstrates the differences between Kubernetes and Calico network policies. You can also join the Calico Users Slack channel for more information.

Related Talks

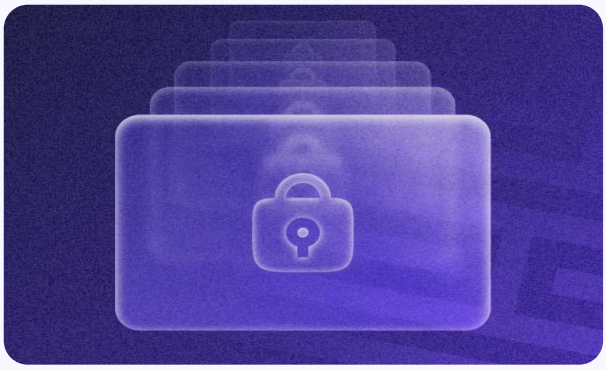
Kubernetes

Manage secrets in K8s using GitOps without compromising security



Kubernetes

Getting started with Kubernetes



Kubernetes

Applying DevSecOps to Kubernetes

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF